Healthcare Lakehouse

End-to-End Data Engineering on Microsoft Fabric

• Medallion Architecture (Bronze / Silver / Gold)
• SCD Type 2 Dimensional Model
• PII Protection (Vault Pattern + DDM)
• Row-Level & Column-Level Security
• Automated Pipeline Orchestration
• DirectLake Power BI Analytics

Alex Rocha | Data Engineer
github.com/alexsrocha82/fabric-healthcare-lakehouse

Table of Contents

1. Architecture Overview

The project implements a complete healthcare data platform on Microsoft Fabric, using the medallion architecture pattern (Bronze / Silver / Gold) within a Lakehouse, a Warehouse for security enforcement, and Power BI for analytics.

All components run within a single Fabric workspace backed by a 60-day trial capacity (F64). Data flows from synthetic CDC generation through the medallion layers, into a secured serving warehouse, and finally to a DirectLake Power BI report.

Component	Fabric Item	Purpose
Lakehouse	lh_healthcare	Bronze/Silver/Gold tables + landing files
Warehouse	wh_healthcare	Serving layer with RLS, CLS, DDM
Notebooks	nb_00 through nb_04	PySpark transformation logic
Pipeline	pl_healthcare_medallion	End-to-end orchestration
Semantic Model	sm_healthcare_analytics	DirectLake model for Power BI
Report	rpt_healthcare_dashboard	Executive dashboard
Environment	env_healthcare	Faker library for data generation

2. Data Flow & Pipeline

Step	Notebook	Duration	What It Does
1	nb_00_generate_synthetic_source	1m 27s	Generate 14 days of CDC data (I/U/D) for 5 entities
2	nb_01_bronze_ingestion	2m 52s	Ingest CSVs as-is, add audit columns, write Delta tables
3	nb_02_silver_transformation	3m 22s	Type cast, DQ checks, PII vault, CDC dedup
4	nb_03_gold_dimensional	3m 08s	Star schema with SCD2 dimensions and fact table

Total pipeline runtime: ~10 minutes end-to-end. Orchestrated by pl_healthcare_medallion with On Success dependencies between each step.

3. Bronze Layer — Raw Ingestion

The Bronze layer ingests CDC change files from the landing zone exactly as they arrived, without any transformation. This creates an immutable, replayable record of all data received.

Key design choices:

• **inferSchema=false:** All columns read as STRING. Prevents Spark from silently converting invalid values to null. Type casting is explicitly handled in Silver.

• **recursiveFileLookup=true:** Reads all partitioned folders (load_date=*) in a single scan.

• **Audit columns:** _source_file (physical file path), _load_date (extracted from partition folder), _ingested_at (ingestion timestamp).

• **Idempotent:** Full overwrite on each run. Incremental variant (Structured Streaming with availableNow trigger) planned for production.

4. Silver Layer — Trusted Data

The Silver layer transforms raw Bronze data into clean, typed, quality-checked datasets. It performs five key operations:

Type Casting: Explicit conversion from string to proper types (Date, Decimal, Boolean, Timestamp) with failure detection. Original values preserved; cast failures flagged in _cast_failed_* columns.

Data Quality: Rule-based checks (not-null, format validation, range checks). Rows failing any rule go to silver_dq_quarantine — not silently dropped. 7 rules for patients, 4 for encounters.

PII Pseudonymization: SHA-256 hash (with salt) of SSN creates patient_hash_id — a deterministic, stable key for linking without exposing the real identifier.

PII Vault Pattern: Sensitive fields (name, SSN, email, phone, address_street) physically separated into silver_patients_pii. The main silver_patients table contains NO PII — only the hash key for joining.

CDC Deduplication: ROW_NUMBER window function partitioned by business key, ordered by _load_date DESC + source_updated_at DESC. Keeps only the latest version per entity. Silver = current state.

5. Gold Layer — Star Schema & SCD Type 2

The Gold layer implements a dimensional model (star schema) optimized for analytics and BI. Two dimensions use SCD Type 2 to preserve historical changes.

Table	Type	Rows	Notes
gold_dim_date	Generated	1,096	Calendar 2023-2025, YYYYMMDD integer key
gold_dim_department	Simple	8	Business key (no SK needed in modern lakehouse)
gold_dim_insurance	Simple	6	Business key, joined via insurance_name
gold_dim_patient	SCD2	1,142	890 current + 252 historical versions
gold_dim_provider	SCD2	54	47 current + 7 historical versions
gold_fact_encounters	Fact	1,685	SCD2 range join: date BETWEEN start AND end

SCD Type 2 Join Pattern:

The fact table joins to SCD2 dimensions using a range condition: encounter_date BETWEEN effective_start AND effective_end. This ensures each encounter links to the dimension version that was active on the day of the encounter, preserving point-in-time accuracy.

6. Security — Defense in Depth

Three complementary layers protect sensitive patient data, each serving a different purpose:

Layer	Where	Mechanism	Protection
1. PII Vault	Lakehouse	Physical table separation	PII doesn't exist in analytics table
2. DDM	Warehouse	ALTER TABLE ADD MASKED	SSN: XXX-XX-3172, Email: jXXX@XXXX.com
3. RLS	Warehouse	SECURITY POLICY + function	Filter encounters by user's region

> *Important: In Microsoft Fabric, RLS/CLS/DDM are enforced only for Viewers. Admins, Members, and Contributors bypass all security policies by design. In production, data consumers must be assigned the Viewer role.*

The Warehouse uses views (not materialized tables) to read directly from Lakehouse Gold tables, avoiding data duplication. The only materialized table is patients_pii, because DDM requires ALTER TABLE which only works on real tables, not views.

7. Key Design Decisions

Surrogate Keys: Modern Approach: Integer SKs were essential in 1990s row-based RDBMS warehouses. In modern lakehouse architectures, Z-ordering, bloom filters, and predicate pushdown make string key JOINs nearly as fast. SKs are used only for SCD2 dimensions (version uniqueness). Simple dimensions use business keys directly.

Silver Dedup vs Gold SCD2: Silver deduplicates to current state (useful for operational queries and DQ). Gold reads from Bronze (full CDC history) to build SCD2 version chains. Both layers serve different purposes and are complementary.

Auto Loader Equivalent: Databricks cloudFiles (Auto Loader) is proprietary and unavailable in Fabric. The equivalent is Spark Structured Streaming with trigger(availableNow=True) and checkpointLocation.

DirectLake Semantic Model: The Power BI semantic model uses DirectLake mode, reading directly from Delta files in OneLake. Zero data duplication, near real-time refresh, no scheduled import needed.

8. Tech Stack & Metrics

Component	Technology
Platform	Microsoft Fabric (Trial F64, 60 days)
Storage	OneLake — Delta Lake / Parquet
Processing	Apache Spark 3.5 (PySpark)
Orchestration	Fabric Data Pipelines

Warehouse	Fabric Warehouse (T-SQL)
BI	Power BI (DirectLake semantic model)
Data Generation	Python + Faker library (en_US)
Security	RLS, CLS, DDM, PII Vault Pattern
Version Control	GitHub + Fabric Git Integration

All data in this project is synthetic, generated by the Faker library. No real patient information was used. The project is fully reproducible: clone the repository, create the Fabric items, and run the pipeline.